Sessions 3, 4 : User groups, new states

5.7 Grading

E Nothing or too few new done beyond the starting point, or does not work at all.

D No comments added, or too few or useless. Added door states locked and unlocked. The ACU still works at door level (no partitions or spaces), like in the starting point code.

C In addition to locked and unlocked states, added spaces and partitions (areas), but not yet user groups and schedules. A user has associated areas and upon a reader request or an area request, you check for each door involved that the user can stay in its "from" and "to" spaces. There is a sufficient number of good comments.

B C + introduced user groups and schedules, access permissions are granted to groups based on areas and schedules.

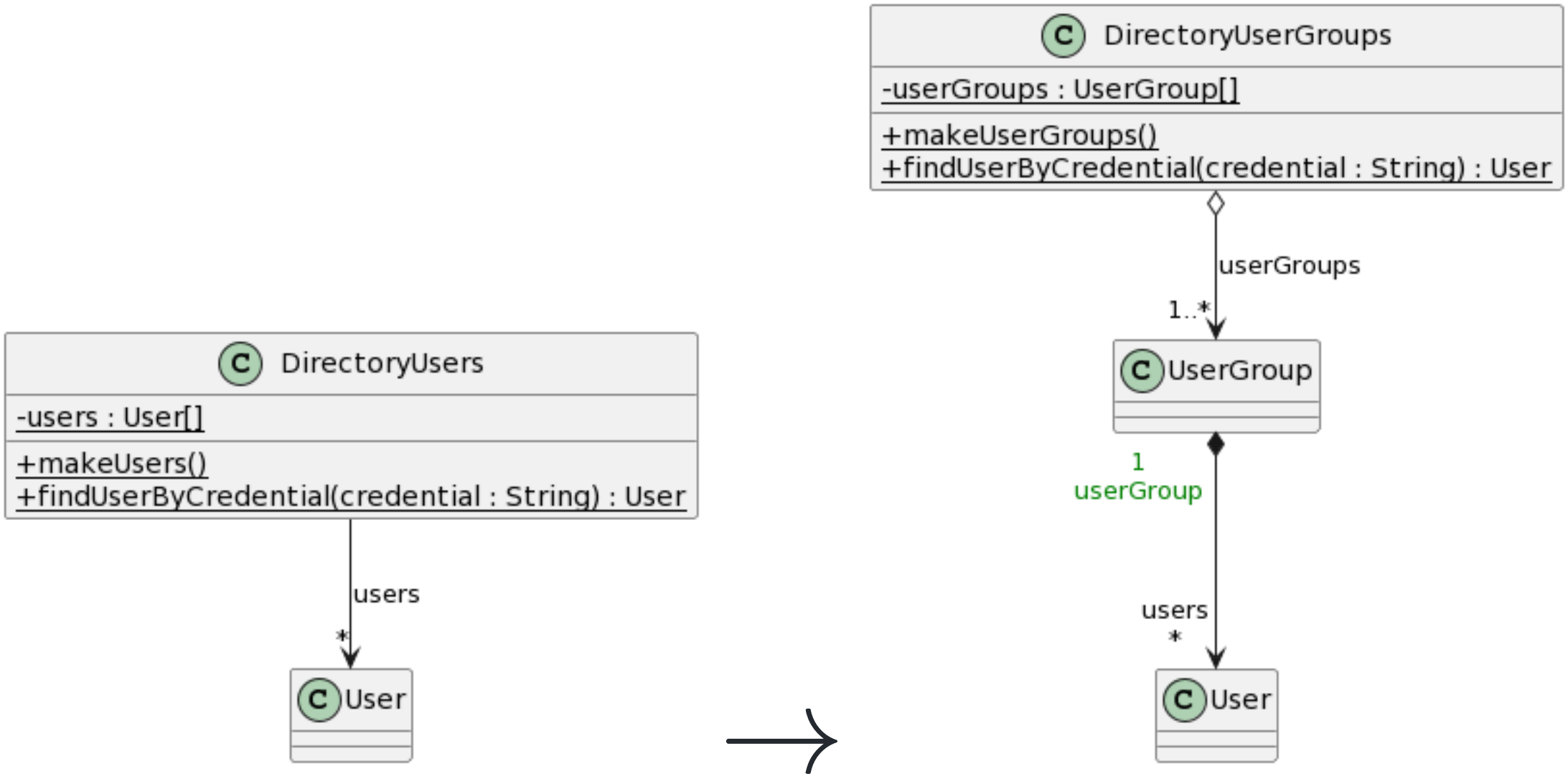
A B + introduced states propped and unlocked shortly which work well.

To all items add that the UML class diagram has to be updated according to the changes in the code.

What is a user group ?

- who : users it contains
- what : actions they can do
- where : on (the doors of) which areas = partitions and/or spaces ?
- when : date interval + weekdays + time interval in a day = `Schedule`

Who has the user groups ? and who instantiates them ?



How to authorize a *reader* request ?

Given a user credential, action name, area id, present date and time

1. retrieve user object
2. retrieve door object (it's a reader request)
3.
```
authorized = user.
  // who
  canSendRequests(now)
  // when : date, time and weekday
  && user.canBeInSpace(door.getFromSpace())
  && user.canBeInSpace(door.getToSpace())
  // where
  && user.canDoAction(action);
  // what
```

- Does the user know if he/she can stay in a space, the date-time can send a request and the actions he/she can do ?

- What if an *area* request instead of reader request ?

- Does the user know if he/she can stay in a space, the date-time can send a request, the actions he/she can do ?

  No, its the user group it belongs to --> a user must know the group it belongs to, see slide #4

- What if an *area* request instead of reader request ?

  Convert it to a list of reader requests. Code already provided.

How to make a clock in Java *running in parallel*

Key classes:

- `LocalDateTime` with static method `now()` returns present time + date
- `java.util.Timer` , extends `implements interface` Thread `with method` run()``
- `java.util.TimerTask` parameter of `Timer.scheduleAtFixedRate()`

Read this.

```
public class Clock {
  private LocalDateTime date;
  private Timer timer;
  private int period; // seconds

  public Clock(int period) {
    this.period = period;
    timer = new Timer();
  }
  public void start() {
    TimerTask repeatedTask = new TimerTask() {
      public void run() { // instance of anonymous class
        date = LocalDateTime.now();
        System.out.println("run() executed at " + date);
      }
    };
    timer.scheduleAtFixedRate(repeatedTask, 0, 1000 * period);
  }
  public void stop() { timer.cancel(); }
  public int getPeriod() { return period; }
  public LocalDateTime getDate() { return date; }
}
```

```
public class Main {
    private static void wait(int seconds) {
        try {
            Thread.sleep(1000*seconds);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

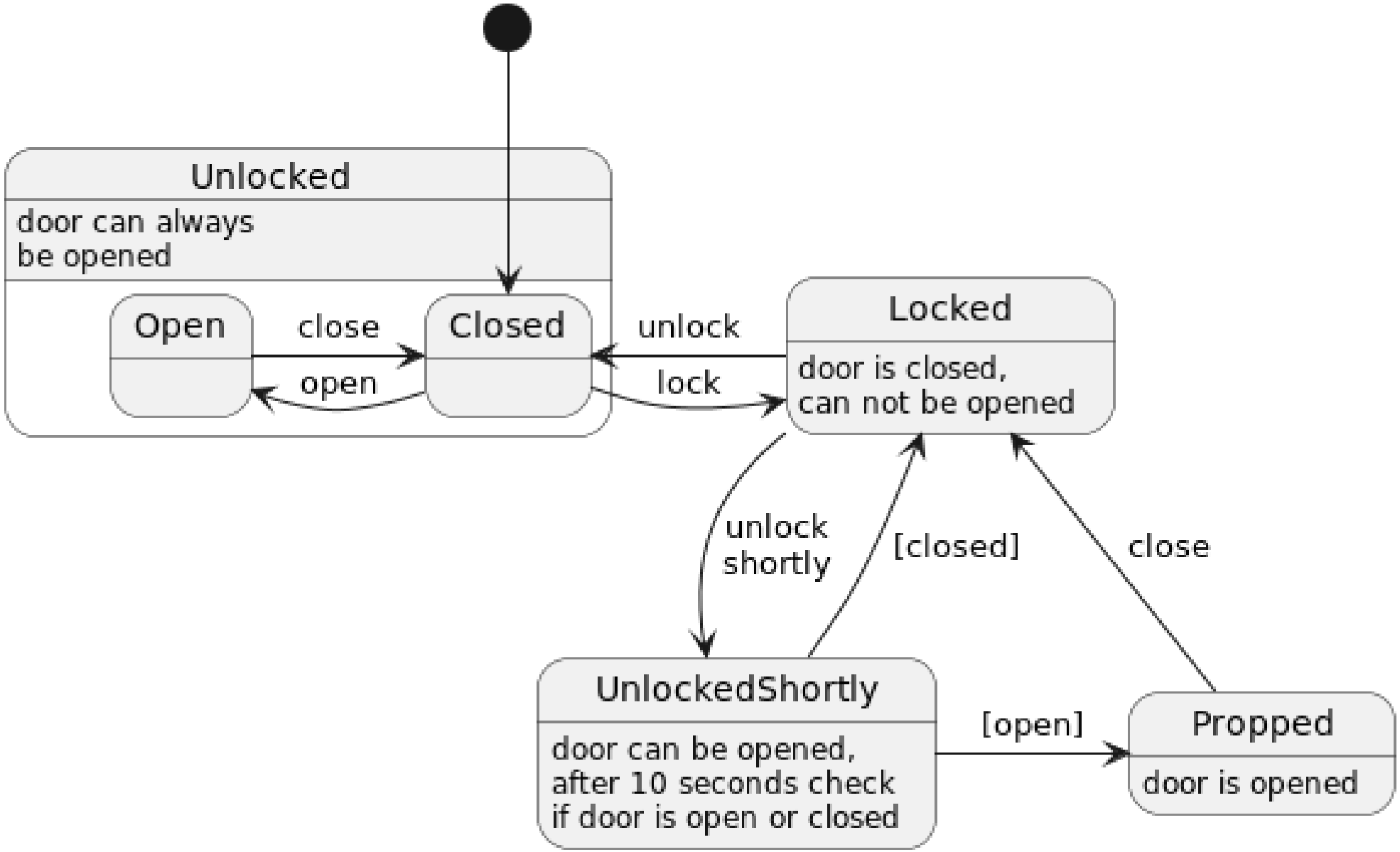
    public static void main(String[] args) {
        final int period = 2; // seconds
        Clock clock = new Clock(period);
        clock.start();
        wait(11);
        // wait for 11 seconds while the clock runs
        // but you can do other things here instead
        clock.stop();
    }
}
```

```
run() executed at 2023-09-29T17:27:06.363275
run() executed at 2023-09-29T17:27:08.342031
run() executed at 2023-09-29T17:27:10.341760
run() executed at 2023-09-29T17:27:12.341569
run() executed at 2023-09-29T17:27:14.341389
run() executed at 2023-09-29T17:27:16.342177

Process finished with exit code 0
```

Why a clock here ?

> ... the door enters into state Unlocked shortly, which means the bolt in the door is switched off and you may open the door and enter (or not). After some seconds, **the ACU automatically** tries to return the door to the locked state.


Unlocked
door can always
be opened
Open
close
Closed
open
unlock
lock
Locked
door is closed,
can not be opened
unlock
shortly
[closed]
close
UnlockedShortly
door can be opened,
after 10 seconds check
if door is open or closed
[open]
Propped
door is opened

Once a door is in `UnlockedShortly` we want the ACU to wait for 10 secs. before checking if door is open or closed, to switch to either `Locked` or `Propped`, while *at the same time* the ACU execution continues : keeps processing newly arrived requests.

How to implement this behaviour ?

Hint: Observer pattern

- which object is observable ?
- which are the observers ?
- who makes an object observer of an observable ?
- when does an observer stop observing ?
- who instantiates a clock ? hint:

```
class X {
  private static final Clock clock = new Clock();
  // a unique clock is shared by all X objects because of static
```

A last detail

The state of a door is a string in the JSON answer, and the name for `UnlockedShortly` the Javascript code (simulator) expects is `"unlocked_shortly"`, not `"unlocked shortly"`:

```
// the size of small squares representing readers of credentials, 10 in the original image
this.colorsOfStates = {"unlocked":"green",
                       "locked":"orange",
                       "unlocked_shortly":"lime",
                       "propped": "red"};
...
// fill the small square representing the reader with the color of its *server* state,
// that is, unlocked, locked etc
paintReader() {
  ctx.fillStyle = this.colorsOfStates[this.state];
  ctx.fillRect (this.xr, this.yr, squareSize, squareSize);
}
```